

Final Assignment

Dimitrios Zografos

Student ID: 03598

September 10, 2026

Course:

Advanced Data Management

Professor:

Konstantinos Liakos

Contents

Introduction.....	3
Part A · Critical Thinking	4
A.1: Why Relational Databases Were Not Enough.....	4
A.2: The CAP Theorem Applied	5
A.3: Column-Family Data Modelling.....	6
A.4: Graph Databases and the Network Topology Problem	8
A.5: Technology Selection Reasoning.....	10
Part B · Implementation	11
Application Overview	11
Deployment and Startup	11
Database Models.....	12
API Structure and Behaviour	13
Seed Data and Repeatability	16
Monitoring and Observability	16
Part C · Measurement	17
C.1: Cassandra Write Throughput vs Consistency Level	17
C.2: Graph Traversal Depth vs Latency	18
C.3: Redis Cache Effectiveness on Dashboard Endpoints	19
C.4: Schema Flexibility in MongoDB vs PostgreSQL JSONB.....	20
Part D · Reflection.....	21
D.1: The Hardest Decision You Made.....	21
D.2: Where the Polyglot Architecture Gets Expensive.....	21
D.3: Graphs in the Wild	22
AI Use Disclosure	23
References.....	23

Introduction

This report presents the final assignment for the Advanced Data Management course in the Department of Electrical and Computer Engineering at the University of Thessaly. It explores the design and implementation of GridSense, a data platform for monitoring an electricity distribution network. The assignment examines how Cassandra, Neo4j, MongoDB, PostgreSQL and Redis support requirements such as sensor ingestion, fault analysis, equipment management and billing. It combines concepts covered in the course with practical implementation and performance evaluation, focusing on the reasoning behind database choices and the trade-offs between consistency, availability, scalability and operational complexity.

Part A · Critical Thinking

A.1: Why Relational Databases Were Not Enough

a. The senior engineer's strongest technical point is that a different data type does not automatically require a different database. Appropriate indexes can support sensor-and-time queries, while time partitioning can restrict searches to relevant periods and allow expired partitions to be removed without deleting readings individually. PostgreSQL also provides transactions and integrity constraints, which are particularly important for billing (Week 1, slides 24–31). Keeping these workloads together would reduce the number of backup procedures, security configurations and failure modes that the team must understand. This matters because a theoretically faster architecture can become less reliable if the organisation cannot operate it properly. I agree that each additional database needs to justify its operational cost through a specific requirement.

b. His argument becomes less convincing when it assumes that indexing and partitioning alone resolve the combined workload. The assignment requires 40,000 sensor events/s normally and 120,000 during storms. Assuming 100 bytes per reading, the normal rate is 4 MB/s, or approximately 345.6 GB and 3.456 billion readings per day. Sustained over 90 days, that would produce about 31.1 TB of raw readings, before indexes, replication and other storage overhead. Independently, the stated minimum of 200 GB/day implies at least 18 TB of retained data.

These numbers do not prove that PostgreSQL cannot handle 40,000 inserts/s. They show why “we can store everything” is insufficient evidence. Durable ingestion also generates WAL and maintains indexes, while billing and dashboard queries compete for resources. Removing whole expired partitions reduces retention-related deletion and VACUUM work, so it would be unfair to assume an unavoidable delete-and-vacuum bottleneck. However, partitioning tables within one server does not itself distribute writes across several servers.

Kleppmann explains that distributed partitioning can spread storage and processing across nodes, but its effectiveness depends on distributing the workload evenly. His sensor example shows how timestamp-based partitioning can concentrate current writes in one hot partition (2017, pp. 199–203). Distributing ingestion across nodes provides a way to add capacity, but the improvement still depends on how evenly the work is shared.

I would favour separating high-volume ingestion from transactional billing, with a specialised topology store justified by the depth-six traversal requirement. The strongest objection remains that PostgreSQL might satisfy the complete workload with suitable hardware and tuning. A representative mixed-workload and failure test could establish that. Until that has been tested, relying on one system to handle storm ingestion alongside critical operational queries carries a substantial risk. This justifies separating some workloads, but does not by itself justify five databases.

c. PostgreSQL-only would be my preferred starting point for a smaller workload: for example, 1,000 readings/s normally, peaks below 5,000/s and 30-day retention. At the same assumed 100 bytes per

reading, that produces approximately 8.64 GB/day and 259 GB of raw retained data. These are illustrative design conditions, not universal PostgreSQL performance limits.

A small team of two or three engineers with PostgreSQL experience, together with shallow topology queries, makes this option more attractive. I would choose it if testing demonstrated the required latency under simultaneous ingestion, billing and topology queries, with sufficient headroom and a tested replication and recovery arrangement. Under those conditions, additional databases could create more operational risk than benefit.

A.2: The CAP Theorem Applied

a. The key question in this disagreement is what happens when the two data centres lose contact. With ONE/ONE, both sides can continue accepting writes and answering reads through their reachable replicas, but a reading acknowledged on one side may be missing from the other. Engineer A accepts that risk. Engineer B would require a majority to respond, so operations on the minority side would stop completing until enough replicas became reachable.

This is where the precise meaning of CAP matters. Gilbert and Lynch's consistency is linearizability: operations must behave like accesses to one copy, respecting real-time order. Their availability requires every request received by a non-failing node to complete, rather than merely achieving good up time or low latency. During a partition, these guarantees cannot both hold. Partition tolerance therefore describes a condition the system must handle, not something either engineer can choose to ignore. Quorums improve consistency by restricting which operations can complete, although they do not automatically establish full linearizability (Gilbert and Lynch, 2002).

b. Assuming $RF = 3$ means three replicas in total, QUORUM for both operations is the smallest symmetric choice that guarantees read–write overlap. The read set must overlap the replicas that acknowledged the write:

$$R + W > RF \Rightarrow 2 + 2 > 3$$

A write acknowledged by replicas A and B must therefore overlap a read from A and C, B and C, or A and B. By comparison, ONE/ONE allows a write to A followed by a read from B, with no overlap.

Thus, $(R=W=2)$ is the minimum symmetric choice. $(R=1, W=3)$ and $(R=3, W=1)$ also work for overlap, but requiring all replicas would make one operation unavailable after a single replica failure. Across two data centres with a 2+1 placement, global quorum remains possible only on the two-replica side during a partition. Separate local quorums do not guarantee cross-centre overlap.

The word “always” needs qualification: this reasoning assumes a completed write, no concurrent conflicting writes, correct version ordering and preservation of acknowledged data. Abadi (2012) explicitly notes that quorum overlap alone does not provide full CAP consistency.

c. For ordinary sensor ingestion, I lean towards Engineer A because B demands stronger freshness than Section 2.2 specifies for counters and dashboard aggregates. These may lag by 30 seconds, while writes must be acknowledged within 50 ms and dashboard P95 latency must remain below 100 ms. Waiting for extra replicas, potentially across data centres, uses part of the latency budget to deliver more up-to-date results than these outputs require.

However, I would not accept A's configuration simply because it offers higher throughput. Eventual consistency does not by itself promise convergence within a fixed 30-second interval (Vogels, 2009). Replication delays must be monitored, and an output exceeding that limit must be treated as degraded rather than presented as current. The single-node failure requirement also needs attention: an acknowledged write could temporarily exist on only one replica, so buffering and replay matter.

My position is therefore that A provides the better starting point, but neither configuration alone proves compliance. I would keep billing and causally ordered fault events outside this justification, since their stricter requirements cannot be relaxed because dashboard aggregates tolerate lag.

d. I find PACELC particularly relevant here because the system operates continuously while partitions are rare. Even with a healthy network, waiting for coordination can delay monitoring requests. The additional question is whether stronger consistency is worth that delay during everyday operation.

Abadi (2012) characterises the original Cassandra design as PA/EL and HBase as PC/EC, while recognising Cassandra's adjustable settings. I would favour Cassandra's lower-consistency options for this ingestion workload because some delayed visibility is explicitly acceptable. HBase's preference for consistency becomes more attractive when acting on stale information would produce an incorrect decision. Rare partitions do not make this choice irrelevant: the normal-operation latency cost is paid repeatedly. I would still require measurements before claiming either system meets the stated targets.

A.3: Column-Family Data Modelling

a. For the first query, `sensor_id` is a suitable partition key because the request already identifies which sensor we need:

```
CREATE TABLE sensor_readings (
    sensor_id text,
    reading_time timestamp,
    metric_type text,
    value float,
    unit text,
    quality_flag tinyint,
    PRIMARY KEY ((sensor_id), reading_time)
) WITH CLUSTERING ORDER BY (reading_time DESC)
AND default_time_to_live = 7776000;
```

The partition key places each sensor's readings together on the responsible replicas, while hashing different sensor IDs distributes their partitions across the cluster. Within each partition, the descending clustering order supports retrieving the newest readings first. The TTL provides 90-day retention.

For the last N readings within a time window, I would use:

```
SELECT * FROM sensor_readings
WHERE sensor_id = ?
```

```
AND reading_time >= ? AND reading_time < ?
LIMIT ?;
```

This targets one partition and a specific timestamp range. I consider it a straightforward fit for pattern (1), although partition growth needs attention, as discussed in (d). The key also assumes one reading per sensor per timestamp: another insert with the same key updates that row rather than storing a separate reading. This query-oriented organisation follows the broader column-family principle of arranging data around expected access patterns and ordered range retrieval (Chang et al., 2006).

b. The same design works against the network-wide query because a timestamp does not identify which sensor partitions to access. A direct attempt would be:

```
SELECT * FROM sensor_readings
WHERE reading_time >= ? AND reading_time < ?
ALLOW FILTERING;
```

This requires reading across partitions and filtering their contents. ALLOW FILTERING allows the query to run but does not provide an efficient lookup path. Alternatively, the application could query every sensor individually, but that creates a large number of requests for each dashboard refresh. A second table gives the dashboard an access path designed around its own query.

c. I would use `sensor_readings_by_bucket`, containing the same reading fields plus `bucket_start` and `shard`, with:

```
PRIMARY KEY (
    (bucket_start, shard),
    reading_time, sensor_id, metric_type
)
```

`bucket_start` is the reading timestamp rounded down to the minute, and `shard` = `CRC32(sensor_id) % 4`. A rolling 60-second window normally overlaps two minute buckets, so the application queries up to eight partitions, applies the exact timestamp bounds and combines the results. This gives the dashboard a known set of partitions instead of searching every sensor partition.

The trade-off is two table writes per incoming reading. At 40,000 events/s, that means 80,000 logical table writes/s; during storms, it becomes 240,000. Replication and storage maintenance add further work. The incoming event rate does not change, but the database must perform roughly twice as many inserts, and partial failures require retries to keep both tables complete.

I would treat four shards as an initial choice that needs testing. With evenly distributed traffic, each current shard receives approximately 10,000 writes/s normally and 30,000 during storms. More shards could spread this load further, at the cost of additional dashboard queries. Even with efficient partition access, 60 seconds contains 2.4 million readings at normal load, so returning every reading is still expensive. Aggregates would be more suitable for the displayed dashboard.

d. I would improve the first table by extending its partition key to (sensor_id, bucket_start), using daily buckets initially. At one reading per second, a sensor produces 7,776,000 readings over 90 days, compared with 86,400 per daily partition. This bounds partition growth while preserving time-range access: the application calculates the relevant buckets and queries them.

However, a large partition is not necessarily a hot partition. One write per second may create substantial history without overloading the replicas. Time bucketing limits partition size and allows successive buckets to map to different replicas, but it does not split simultaneous writes from one sensor. If a sensor's write rate remained excessive, an additional shard derived from the reading timestamp could spread those writes, with results merged across the shards when reading. I would accept that extra read complexity only when the write load justified it.

A.4: Graph Databases and the Network Topology Problem

a. I would favour Neo4j because fault analysis repeatedly follows supply connections, and the number of steps depends on where the fault occurs. SQL can answer the same questions, but an explicitly expanded traversal becomes longer as the depth increases. Assuming a relational connections table containing the supply links, a two-hop downstream query would be:

```
SELECT DISTINCT c2.to_id AS node_id
FROM connections c1
JOIN connections c2 ON c1.to_id = c2.from_id
WHERE c1.from_id = :node_id
  AND c1.active = TRUE
  AND c2.active = TRUE;
```

The equivalent Cypher query is:

```
MATCH p = (origin {node_id: $node_id})
  -[:FEEDS|SUPPLIES|CONNECTS_TO*2]->(downstream)
WHERE all(r IN relationships(p) WHERE r.active = true)
RETURN DISTINCT downstream.node_id AS node_id;
```

At depth two, the SQL uses one self-join between two edge-table aliases. At depth six, it needs five self-joins between six aliases. In Cypher, *2 becomes *6, while *1..6 retrieves nodes at any depth up to six. I find this easier to maintain when traversal depth varies. A document database would require repeated reference lookups or duplicated topology data, making shared connections and alternative routes harder to manage.

The discussion in *Graph Databases*, Chapter 2, supports this preference for connected queries. However, recursive SQL can also express variable-depth traversal, and shorter Cypher does not automatically mean

faster execution. The advantage is that Neo4j is organised around relationship traversal, although branching paths still increase the work required.

b. For the network model, I propose four main labels: GridSupplyPoint, Substation, Transformer and SmartMeter. Each node would have a common node_id and a name. Supply points and substations would include voltage and location, transformers would include rating_kVA, and meters would include meter_id and premise_id.

The relationships would be FEEDS from supply points to substations, SUPPLIES from substations to transformers, and CONNECTS_TO from transformers to meters. Feeder or cable identifiers, connection length and active status belong on these relationships. These properties describe the connection: a transformer could have two incoming cables with different lengths and switching states. Storing one value on the transformer would lose that distinction. This follows Chapter 4's guidance on separating entities from the connections and properties that describe their structure.

Inactive backup links should remain in the graph so restoration searches can find them. I would still distinguish a possible route from a usable one: it must avoid the failed equipment and have sufficient capacity. Likewise, a downstream node should only be classified as disconnected after checking whether another live source still reaches it.

c. Receipt of a fault should not depend on Neo4j being ready to accept a write. During a leader election, writes can temporarily fail or wait because the cluster needs an elected leader and a reachable majority. Some reads may continue, but they may not include the latest fault-related changes. This is the practical availability cost of protecting consistent writes.

My proposed solution would accept faults into an independently available, durable replicated queue or log before acknowledging them. The system could issue an immediate basic alert identifying the relay, while a consumer updates Neo4j and calculates the detailed impact once the leader is available.

Each event would have a unique identifier so retries do not create duplicate processing. Processing would only be marked complete after the result and notification were durably recorded. I consider a visible "fault received, impact analysis pending" state preferable to either losing the event or presenting an outdated analysis as current.

d. A small, bounded collection of reading nodes can be useful when their relationships help answer specific questions. For example, 100 sensors reporting every 15 minutes produce 9,600 readings per day, or 288,000 over 30 days. That is a plausible workload to evaluate, rather than a guaranteed capacity limit. At 40,000 readings/s, however, the proposal creates 3.456 billion reading nodes and at least as many relationships every day. Over 90 days, that reaches approximately 311 billion nodes and 311 billion relationships. Ingestion, indexing and deletion would compete with the topology queries that justify using the graph database.

For that workload, I would keep full telemetry in the time-series store and use the graph for topology, selected fault events and useful summaries. The deciding factor is whether representing each reading as a connected node adds enough analytical value to justify its storage and maintenance cost.

A.5: Technology Selection Reasoning

1. Cassandra. The forensic query identifies a particular fault event and then replays its operations in order, which fits Cassandra's partitioned and sorted storage. Partitioning by `fault_id` and clustering by an ascending event sequence would keep the relevant operations together for sequential retrieval. The sequence must preserve causal dependencies at ingestion, since timestamps alone cannot reliably establish causality across devices. I would enforce append-only behaviour through a controlled ingestion service using conditional inserts to prevent overwrites, with no automatic expiry or direct modification access for ordinary users. Corrections would be recorded as new events so the original sequence remains intact.

2. Neo4j. The query follows a specific two-hop path: from the flagged meter back to its transformer, then out to the other meters connected to that transformer. Their consumer or premise identifiers can then be returned, excluding the flagged meter and removing duplicates where necessary. This makes the search local to the relevant connections rather than requiring an examination of every consumer. I would filter for active connections so that an inactive backup route does not incorrectly imply that two meters currently share the same supply.

3. Redis. The overload status can be stored under one key per transformer, such as `overload:TX_001`, with a five-second expiry. Repeated lookups by transformer identifier suit an in-memory key-value store, and the 22,000 small status values do not require complex querying. I would set the value and expiry in the same operation so every update has the intended lifetime. An expired or missing key should mean that the status is unknown, rather than that the transformer is operating normally.

4. MongoDB. The new meter's 40 manufacturer-specific fields can be stored in a nested document alongside its common identifiers, without changing the structure of every other meter record. This suits an access pattern where the application retrieves a meter's details together, including fields that vary between models. I would still validate the common fields and each model's expected data types, since a flexible schema should not lead to inconsistent records. PostgreSQL JSONB could also handle this requirement, but MongoDB provides a straightforward document model for equipment with substantially different structures.

5. PostgreSQL. The deciding requirement is that billing changes for all 1.2 million accounts must either commit together or leave no partial result. SQL can combine consumption records with progressive tariff bands, surcharges and time-of-use adjustments, using exact decimal arithmetic for monetary calculations. I would use a consistent set of inputs and apply the invoices and balance adjustments within a transaction covering the complete billing run. The transaction's size requires careful planning for locking, transaction-log volume and recovery, but committing independent account batches would not satisfy the stated all-or-nothing requirement.

Part B · Implementation

Application Overview

GridSense is a backend application for managing information about an electricity distribution network. A FastAPI REST interface provides access to sensor measurements, network topology, equipment metadata, consumer accounts and alerts. The application uses shared identifiers to associate records across databases: a smart meter can share its identifier with its sensor readings and equipment document, while its `premise_id` links to a consumer account. These associations depend on consistent application data. Foreign-key constraints do not extend across the separate databases.

Deployment and Startup

The environment is defined in `docker-compose.yml` and contains the following services:

Service	Technology	Exposed ports
api	FastAPI, Python 3.11 and Uvicorn	8000
timeseries-db	Cassandra 4.1	9042
graph-db	Neo4j 5 Community	7474 (browser), 7687 (Bolt)
catalog-db	MongoDB 7	27017
billing-db	PostgreSQL 15	5432
cache	Redis 7 Alpine	6379
prometheus	Prometheus	9090
grafana	Grafana	3000

The containers communicate through the `gridsense_net` network using Compose service names. Named volumes retain database files when containers are replaced. The API image is built from `python:3.11-slim`, installs `api/requirements.txt` and starts Uvicorn with `main:app`.

Before startup, `.env.example` is copied to `.env` and its placeholder credentials are replaced. The local `.env` file supplies passwords and connection settings and is excluded from Git tracking. The API and seed services load it directly, while Compose passes the relevant variables to the database services. From the repository root, the system is started with:

```
docker compose up --build
```

Database preparation follows this sequence:

1. PostgreSQL executes `postgres/init.sql` when its data volume is first initialised.
2. Cassandra passes node-status and CQL connectivity checks, then the one-shot `cassandra-init` service executes `cql/init.cql`.
3. The seed service waits for Cassandra initialisation and healthy PostgreSQL, MongoDB and Neo4j services. It then runs the data-loading process described in Section 5.
4. The API starts after seeding completes successfully and Redis is healthy.

This deployment contains one instance of each database and represents a development environment, without a replicated failover configuration.

Database Models

Cassandra

The gridsense keyspace uses SimpleStrategy with replication factor 1. Sensor records contain sensor_id, reading_time, metric_type, value, unit and quality_flag. Their two storage layouts support different access patterns:

Table	Partition key	Clustering columns	Purpose
sensor_readings	sensor_id	reading_time DESC	Sensor-specific history, newest first.
sensor_readings_by_bucket	(bucket_start, shard)	reading_time DESC, sensor_id, metric_type	Time-based access across sensors.
relay_events	feeder_id	event_time ASC (TIMEUUID)	Relay-operation records ordered within a feeder.

For the bucket table, bucket_start is the measurement timestamp rounded down to a UTC minute, and CRC32(sensor_id) % 4 selects one of four shards. Both sensor tables have a default TTL of 7,776,000 seconds, equivalent to 90 days from the write. The main table permits one row per sensor and timestamp, so another measurement with that key updates the existing row. Relay records include relay_id, event_type, fault_type and current_kA, with no default expiry. The relay table is defined, but application-level relay ingestion, immutability enforcement and causal-order validation are not implemented.

Neo4j

The topology uses four node labels. Every seeded node has a common node_id and name, together with properties specific to its equipment type:

Label	Type-specific identifier	Other properties
GridSupplyPoint	gsp_id	Voltage and region.
Substation	substation_id	Voltage, latitude, longitude and commissioning year.
Transformer	asset_id	Rating, manufacturer, model, installation and inspection dates.
SmartMeter	meter_id	Premise identifier, tariff class and phase.

Uniqueness constraints cover each label's type-specific identifier. FEEDS connects supply points to substations and carries feeder ID, voltage and length. SUPPLIES connects substations to transformers and carries cable ID and distance. CONNECTS_TO connects transformers to meters. All three relationship types carry an active property, allowing normal and inactive backup connections to coexist. Extending this model

to the full 26,000-node network means adding records using the same labels and relationships. The current API searches by the common `node_id`, so that expansion should include an indexed lookup strategy for this identifier and traversal tests against realistic branching and backup paths. The existing type-specific constraints do not provide an index for an unrestricted `node_id` lookup, and the small seed does not establish full-network performance.

MongoDB

Equipment metadata is stored in the `equipment` collection. Documents share `asset_id` and `equipment_type`, while additional fields vary by device. Smart-meter records include firmware, rated voltage, phase and communication protocol. Transformer records include electrical ratings, cooling type and oil capacity. Switchgear records include rated current, breaking capacity, insulation and pole count. This allows different equipment structures in the same collection. The API models accept extra fields rather than imposing one fixed document shape on every equipment type.

PostgreSQL

Billing data is divided between accounts and invoices. The account primary key is `premise_id`, alongside customer name and contact details. Invoices have a generated `invoice_id`, a foreign key to the account, an amount, payment status and `due_date`. Amounts use `DECIMAL(10,2)` with a database check preventing negative values. Outstanding balances are derived from invoice records rather than stored as a separate account balance. The operational billing schema uses ordinary relational columns. It does not contain the JSONB field mentioned in the service description of the brief.

Redis

Redis holds sensor summaries and alert state. Summaries use string keys named `sensor_summary:{sensor_id}` with a 30-second expiry. Active alerts are JSON values in the `alerts:active` hash, while `alerts:sequence` supplies incrementing alert identifiers. The alerts Pub/Sub channel carries notifications to connected subscribers. Active-alert entries have no configured TTL. The hash provides retrievable current state, whereas Pub/Sub provides live delivery. The implementation has no durable consumer acknowledgement or notification replay mechanism.

API Structure and Behaviour

The application separates routing, validation and database access:

Location	Responsibility
<code>api/main.py</code>	Creates the FastAPI application, registers routers and manages the connection lifespan.
<code>api/routers/</code>	Contains <code>sensors.py</code> , <code>grid.py</code> , <code>equipment.py</code> , <code>billing.py</code> and <code>alerts.py</code> .
<code>api/models/</code>	Contains Cassandra, graph, MongoDB and PostgreSQL request/response models. The alert input model is defined in its router.

api/db/		Holds one connection module for each database.
api/Dockerfile	and	Define the API runtime and Python dependencies.
api/requirements.txt		

Connections open before requests are served and close during shutdown. Neo4j uses AsyncGraphDatabase, MongoDB uses Motor, PostgreSQL uses an asyncpg pool of one to five connections, and Redis uses redis.asyncio. Cassandra uses cassandra-driver, with connection setup and most query execution moved to worker threads through asyncio.to_thread. Pydantic validates inputs, including sensor quality flags from 0 to 2, type-specific graph properties and positive invoice amounts. Graph node and relationship models reject unexpected fields. Equipment models allow them. Interactive API documentation is available at <http://localhost:8000/docs>.

REST Endpoints

Method	Path	Implemented behaviour
POST	/sensors/readings	Accepts a reading or batch and writes each measurement to both sensor tables. Rejects empty batches.
GET	/sensors/{sensor_id}/readings	Returns newest-first readings with optional from_time. Limit defaults to 100 and accepts 1–10,000.
GET	/sensors/{sensor_id}/summary	Returns a cached summary or builds one from the latest reading and previous hour's measurements, calculating count, average, minimum and maximum per metric.
GET	/grid/fault-impact/{node_id}	Returns downstream nodes, their identifiers, types, names, minimum traversal depth and total count.
GET	/grid/restore-paths/{node_id}	Returns supply-point paths with source ID, ordered node IDs, hop count and active status.
POST	/grid/nodes	Creates a validated node and checks for an existing common node_id.
POST	/grid/relationships	Creates a relationship after checking endpoint existence, permitted node-label pairing and an existing link of the same type.
GET	/equipment/{asset_id}	Retrieves the equipment document, excluding MongoDB's internal _id.
POST	/equipment	Registers a document after checking whether its asset ID already exists.
PATCH	/equipment/{asset_id}	Applies supplied fields using \$set. Rejects empty updates and changes to asset_id or _id.
GET	/billing/account/{premise_id}	Returns account details and the sum of its unpaid invoices, or zero when none are unpaid.
POST	/billing/invoice	Checks account existence and inserts the submitted invoice within a PostgreSQL transaction.

GET	/alerts/active	Reads the active-alert hash and returns its entries and count.
POST	/alerts/publish	Generates an alert ID, stores an ACTIVE alert and publishes its JSON payload.

Sensor ingestion performs the two table writes sequentially for each reading. They are separate operations, so a partial failure can leave the tables out of sync. The bucket table is populated but has no dedicated retrieval route. Summary cache misses read Cassandra and populate Redis. New readings do not invalidate an existing cached summary.

Both graph queries accept `max_depth`, defaulting to 6 and restricted to 1–10. The validated integer is inserted into the Cypher traversal bound, while node identifiers are passed as parameters. Queries run in asynchronous session context managers and return Pydantic response models. Fault impact groups paths by downstream node and uses `min(length(path))` for depth. It follows inactive links as well as active ones and does not test whether another source still supplies the node, so its result represents downstream reachability rather than a confirmed outage set. Restoration marks a path active only when every relationship is active and sorts active paths first, followed by hop count and source ID. It lists candidate paths without excluding specified failed equipment or checking spare capacity.

Graph queries distinguish an unknown node, which returns HTTP 404, from an existing node with no matching paths, which returns an empty result. Creation checks return HTTP 409 for detected duplicates. Invalid relationship pairings return HTTP 400. These application checks do not constitute a single atomic check-and-create operation. Equipment duplicate checks likewise operate at application level, without a unique `asset_id` index defined by this code.

Invoice creation commits the submitted amount, status and due date for one account. It does not calculate consumption-based tariffs or perform a monthly transaction across all accounts. Alert publication accepts the node, tag, alarm type (HH, H, L or LL), limit, value, priority and message. It adds a sequence-based ID and UTC timestamp, then stores and publishes the alert through separate Redis commands. Sensor ingestion does not automatically evaluate alarm thresholds or trigger this endpoint.

Additional Endpoints

- POST `/alerts/{alert_id}/clear` removes an active alert and publishes a CLEARED event with an updated timestamp. An unknown alert returns HTTP 404.
- GET `/health` returns a basic application response without checking every database again.
- GET `/metrics` exposes FastAPI request metrics. Prometheus collects them every five seconds, and Grafana loads its configured datasource and dashboard automatically.

Seed Data and Repeatability

The master loader is `scripts/seed.py`, executed as `python scripts/seed.py` inside the configured seed container. It calls the Cassandra, Neo4j, MongoDB and PostgreSQL loaders in sequence. The Neo4j loader executes `neo4j/import/seed.cypher`, including its uniqueness constraints and topology creation. MongoDB's collection is created by its first document writes. To rerun loading with the container's dependencies and environment settings, use:

```
docker compose run --rm seed
```

Direct host execution of the Python command requires the Python dependencies and database environment variables to be available. The script does not load `.env` itself.

Database	Dataset produced on a clean seed	Behaviour on a sequential rerun
Cassandra	30 sensors × 2,000 measurements: 60,000 readings in each sensor table.	Fixed sensor IDs and timestamps target the same primary keys. Rewrites refresh TTLs without adding duplicate rows.
Neo4j	2 supply points, 10 substations, 40 transformers and 200 meters: 252 nodes, with 250 main and 6 backup relationships.	MERGE reuses existing seeded nodes and relationships.
MongoDB	30 equipment documents: 10 smart meters, 10 transformers and 10 switchgear units.	<code>replace_one(..., upsert=True)</code> matches <code>asset_id</code> and replaces or inserts each seed document.
PostgreSQL	100 accounts and one sample invoice per account.	Accounts use ON CONFLICT. Invoices are inserted only when the same premise and due date are absent.

Cassandra measurements use five-second intervals from a fixed start time of 1 September 2026, with deterministic voltage, current and power-factor values and a mixture of quality flags. Consequently, the seed provides historical data rather than a continuously current stream. Testing the previous-hour summary with populated statistics requires recent readings. The graph includes inactive backup links at each level of the hierarchy, and its meter and premise identifiers overlap with the corresponding sensor and billing seed records. Redis receives no seed dataset. its keys are populated through application use.

Monitoring and Observability

GridSense also includes a Prometheus and Grafana monitoring stack. The `/metrics` endpoint using `prometheus-fastapi-instrumentator`, allows request counts, errors and request latency to be collected. Prometheus runs as a separate Docker service and stores the collected metrics, while Grafana is provisioned as another service for visualising them through a dashboard. This was added to provide a simple view of API behaviour and to satisfy the observability requirement beyond basic application logging.

Part C • Measurement

C.1: Cassandra Write Throughput vs Consistency Level

benchmarks/c1/locustfile.py inserts voltage readings directly into sensor_readings. C1_CONSISTENCY selects ONE, LOCAL_QUORUM or ALL. Each configured user writes under a sensor identifier, using the current UTC timestamp and the same measurement fields. The workload has no deliberate waiting period between writes. Successful operations and exceptions are reported to Locust, and the saved results contain four runs for each level. The latency columns are averages of the four per-run percentiles.

Consistency level	Mean write operations/s	Mean run P50	Mean run P95	Recorded errors across four runs
ONE	2,075.67	12.00	26.00	0
LOCAL_QUORUM	2,051.86	12.00	25.50	0
ALL	2,102.78	12.00	25.25	0

With RF = 1, ONE, LOCAL_QUORUM and ALL each require an acknowledgement from the same single replica, so ALL has no additional replica to wait for. The highest and lowest mean throughput differ by about 2.5%, while individual runs of ONE range from 1,917.97 to 2,187.68 operations/s. ONE's fourth run pulled its average down, recording the lowest throughput and a P95 of 31 ms. That run also included a maximum latency of 714 ms, although this single slow operation cannot explain the whole performance difference. During testing, a warning indicated CPU usage around 90%, suggesting that CPU pressure probably contributed to the variation and may have limited the generated workload. These results therefore do not show that ALL is inherently faster than ONE.

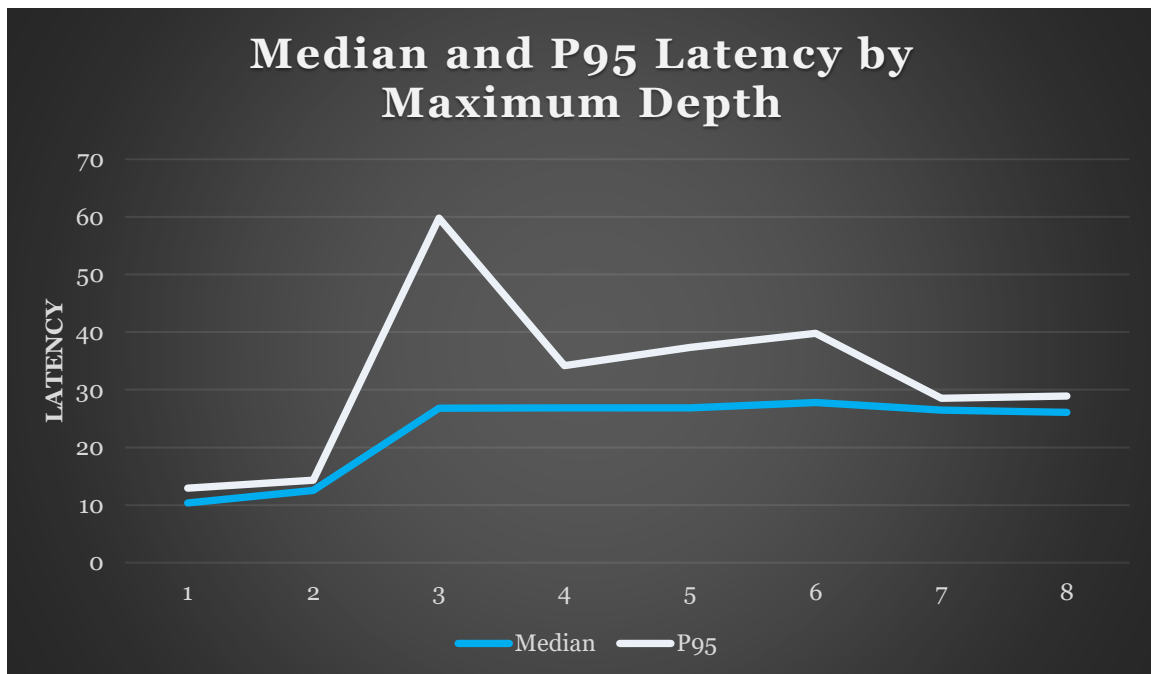
With three nodes and RF = 3, the difference would become more meaningful. ALL would wait for all three replicas, whereas QUORUM could acknowledge the write after two responses. A slow third replica could therefore delay ALL without delaying QUORUM, and an unavailable replica would prevent ALL writes from completing while QUORUM could continue. Under those conditions, lower latency and higher throughput would generally be expected from QUORUM, although the size of the difference would depend on the workload and replica response times.

For production, I would start with LOCAL_QUORUM, assuming three replicas in the local data centre. The main reason is that an acknowledged reading would have reached two replicas, reducing dependence on a single node while still allowing ingestion to continue when one replica is unavailable. The dashboard can tolerate delayed values, but that does not mean acknowledged readings can be lost. ONE becomes more attractive if a separate durable ingestion log can replay missing readings and measurements show that waiting for two replicas prevents meeting the latency target. Since the current implementation has no such replay mechanism, LOCAL_QUORUM offers the better starting balance.

C.2: Graph Traversal Depth vs Latency

benchmarks/c2/benchmark.py calls the fault-impact endpoint for GSP_NORTH, varying max_depth from 1 to 8. Each setting receives 30 sequential requests against the same seeded graph. Timing covers the complete HTTP call, including query execution, result processing and response transfer. The source reaches 250 distinct downstream nodes in the seed, exceeding the experiment's minimum of 20.

Maximum depth	Median latency	P95 latency
1	10.36	12.93
2	12.53	14.31
3	26.81	59.78
4	26.85	34.17
5	26.87	37.38
6	27.79	39.77
7	26.45	28.53
8	26.09	28.93



Neither the median nor P95 crosses 200 ms at any tested depth. I would set the default max_depth to 3 for the current implementation because it reaches every downstream layer, including meters. Higher settings return no additional nodes in this topology, and the measurements show no clear benefit from increasing the limit. This choice follows the graph's structure rather than proving that three is a performance optimum. If the production model introduces deeper connections, the limit should be reconsidered and tested so that relevant nodes are not silently excluded.

Latency rises through depth 3 and then plateaus near 27 ms median. From GSP_NORTH, the distinct result grows from 10 nodes to 50 and then 250, after which no deeper nodes remain. This does not establish linear, polynomial or exponential complexity. The query explores variable-length paths and groups them using `min(length(path))`, so additional branching and alternative routes can increase its work even without increasing the number of returned nodes. The measured latency also includes HTTP overhead and processing larger responses.

I am not so confident in extending this behaviour to 26,000 nodes. A larger topology could introduce deeper paths, more branching and larger responses, while concurrent requests would compete for resources. The current query also searches by `node_id`, whereas the seed's uniqueness constraints cover type-specific identifiers. An indexed lookup strategy for `node_id` and tests on a representative full-size graph would be needed before trusting the same latency figures. The small benchmark supports the current prototype, but does not prove production scalability.

C.3: Redis Cache Effectiveness on Dashboard Endpoints

`benchmarks/c3/benchmark.py` measures the summary endpoint for SM_00001. An initial untimed request fills the cache, followed by 500 sequential warm-cache requests. Before each of the next 500 requests, the script deletes `sensor_summary:SM_00001`. This deliberately forces misses rather than waiting for the 30-second TTL to expire between batches. The deletion occurs outside the timed HTTP request. Each cold request still performs the usual cache lookup, Cassandra retrieval, summary calculation and Redis write.

Batch	Timed requests	P50	P95	P99	Measured cache hit rate
Warm cache	500	3.97	6.22	10.66	100%
Forced cache miss	500	12.60	18.47	23.60	0%

The warm-cache median was 3.97 ms, compared with 12.60 ms for forced misses, a reduction of approximately 68.5%. I cannot attribute a specific fraction of this difference to network overhead or database execution because I measured complete HTTP requests without timing those stages separately. On a miss, the endpoint prepares and executes two Cassandra queries, processes their results and stores the summary in Redis. The additional latency includes all this work and its communication costs. At the time I performed this benchmark, there were no readings within the previous-hour window because the seed timestamps were older. With readings in that window, I would expect misses to become slower because the code retrieves all matching rows and calculates their statistics in Python. My results therefore show the cost of an empty hourly summary, rather than a busy sensor's full hour of data.

All 1,000 measured requests target SM_00001. The script first warms its cache, sends 500 requests, then deletes the same key before each of another 500 requests. This workload provides no evidence that another eviction policy would improve hit rates because it does not test competing sensor keys, and changing the policy cannot prevent explicit deletion. I would keep `allkeys-lru` for now. Comparing it with `allkeys-lfu` would require many sensors with different request frequencies and enough memory pressure

to trigger eviction. Compose does not configure an explicit maxmemory limit, while the benchmark records neither evictions nor hit/miss counters, so actual hit rates cannot be verified from these results.

C.4: Schema Flexibility in MongoDB vs PostgreSQL JSONB

The comparison uses the same 30 equipment records in MongoDB and in a benchmark-specific PostgreSQL table, `equipment_jsonb`. `benchmarks/c4/seed_postgres_jsonb.py` creates an `asset_id` primary key and a metadata JSONB column, then inserts or updates the matching equipment records. This table is additional benchmark storage and is separate from the application's billing schema.

`benchmarks/c4/benchmark.py` executes each query once before measurement. It then runs each query 10 times per database and reports the arithmetic mean. The script compares asset-ID sets for the first two queries and grouped counts for the third.

Query	MongoDB mean	PostgreSQL mean
Q1: firmware version starts with 3.	1.778	0.504
Q2: SmartMeter with voltage above 230	1.645	0.530
Q3: count equipment by type	1.660	0.475

For these 30 equipment records, the performance difference has little practical importance for occasional catalogue queries. PostgreSQL's recorded means are approximately 3.1-3.5 times faster, but the absolute saving is only 1.12-1.27 ms per query. The benchmark tests two filters returning identifiers and one aggregation counting equipment by type, with each query warmed up before ten measured executions. At this size, communication and driver overhead can contribute substantially to the timings. I would therefore not choose between MongoDB and PostgreSQL JSONB based on this difference alone.

The difference would start to matter when a larger catalogue or more simultaneous requests makes query latency or resource use a constraint. The scripts create no secondary indexes on the tested fields, so increasing the dataset would increase the work needed to examine records and calculate grouped counts. However, this benchmark cannot identify a specific size at which the difference becomes significant or establish that PostgreSQL would remain faster. I would extend the test to 10,000, 100,000 and 1,000,000 records, using suitable indexes for each database and concurrent clients. These would be test sizes, with the decision based on measured response times and resource use rather than extending the current timing ratio to larger datasets.

Part D • Reflection

D.1: The Hardest Decision You Made

The hardest technical decision I made was how to represent backup supply paths in Neo4j. The assignment required a `/grid/restore-paths/{node_id}` endpoint, but it did not explain how active and backup paths should be modelled. This was one of the points where I had to make my own design decision. I kept the existing relationship types, such as FEEDS, SUPPLIES, and CONNECTS_TO, and added an active property. Normal operating paths use `active = true`, while redundant links use `active = false`. I also added a second grid supply point and inactive connections so the endpoint could actually discover an alternative route.

After implementing this, I considered using a separate relationship type such as BACKUP_SUPPLIES. I decided against it because that would mix the physical type of a connection with its current operating state. A supply path can be inactive now and become active after switching, so SUPPLIES plus an active state seemed more flexible. What would have helped me decide faster was a clearer model of switching devices and restoration logic. In a real network, a boolean would probably not be enough because breaker state, switch position, equipment availability, capacity, and voltage constraints can all affect whether a path is usable.

If I were doing it again, I would design the switching and operational-state model first and then build the restore-path query around it. For this prototype, active/inactive was a simple and effective solution, but a production system would need a more detailed representation.

D.2: Where the Polyglot Architecture Gets Expensive

The first place where the polyglot architecture creates a real burden is equipment identity across MongoDB, Neo4j, and Cassandra. The same physical device can appear as an equipment document in MongoDB, as a node in the Neo4j topology, and as the source of readings in Cassandra. These systems do not share a transaction, so an update may succeed in one database and fail in another. For example, a meter could be registered in MongoDB but not connected correctly in Neo4j, leaving the system in an inconsistent state. In production this would require additional synchronisation logic, retries, validation, and monitoring.

The second burden is the operational cost of maintaining five different database systems. GridSense is already defined as a complete system of about 26,000 topology nodes, so I would not assume that its size will continue growing indefinitely. At that scale, I think some of the specialised databases may provide less benefit than the additional complexity they introduce. Each engine requires separate deployment knowledge, monitoring, backup procedures, upgrades, security configuration, and troubleshooting skills. To reduce this burden, I would test whether PostgreSQL could absorb most workloads except the high-volume sensor storage. I would keep Cassandra because the 40,000 events/s normal ingestion rate and 120,000 events/s storm peak are the most demanding part of the system and clearly justify a specialised distributed write-oriented database. PostgreSQL could potentially handle billing, equipment metadata

through JSONB, and possibly the 26,000-node topology using relational tables and recursive queries. This is especially reasonable because our C.4 experiment showed that PostgreSQL JSONB handled the same equipment queries correctly and performed well at the tested scale.

I would not automatically keep Redis either. I would first test the simplified Cassandra-plus-PostgreSQL architecture against the required latency targets. If it met them, I would prefer the simpler design. The trade-off is that this approach gives up some of the natural modelling and performance advantages of MongoDB, Neo4j, and Redis. However, for a fixed-size GridSense deployment, I would accept some loss of specialisation if it significantly reduced operational complexity and cost.

D.3: Graphs in the Wild

Before this course, I mostly thought about industrial data as values coming from PLCs through communication drivers into an HMI or historian. The engineering work was mainly to configure the mapping: which PLC address or protocol item corresponds to which tag, how it should be named, and where it should be used. That works well for live and historical data, but the relationships between all these elements are usually spread across driver configurations, naming conventions, drawings, spreadsheets, and documentation.

Working with Neo4j and Cypher changed the way I think about that information. I realised that the relationships themselves can be treated as useful data, rather than just configuration. A graph model makes it natural to represent how equipment, PLCs, communication addresses, tags, historian points and higher-level applications are connected.

A concrete example would be a semantic model of an industrial plant. The underlying communication could be OPC UA, Modbus, or another industrial protocol. This is especially useful for protocols such as Modbus, where an address such as a register number does not contain any real information about what the value represents. An engineer could define in the graph that a specific address represents the outlet temperature of a pump, that the pump belongs to a cooling system, that the signal is read from a certain PLC, and that it is consumed by an HMI, historian or MES application.

This would create a structured information layer for the whole factory. Engineers adding a new application would not need to rediscover the meaning and location of every tag from different configuration files. They could query the model to find all signals belonging to a machine, which PLC provides them, which applications use them, or what systems depend on a particular device.

I also think this could become particularly useful for AI-based industrial applications. Instead of giving an AI system thousands of tag names and raw addresses with little context, the graph could provide the relationships and meaning behind those values. I would not use the graph database to replace the PLC communication or historian, but I would now seriously consider it as a shared semantic layer that makes the existing industrial data much easier to understand and reuse.

AI Use Disclosure

I used ChatGPT as a supporting tool during the development and preparation of this project.

For Part B, I used ChatGPT interactively while implementing the different components. Typical prompts asked for explanations of Cassandra data modelling, Neo4j and Cypher, Redis caching and Pub/Sub, MongoDB and PostgreSQL usage, FastAPI structure, Docker Compose configuration, and errors encountered while running the system. AI sometimes suggested modifications, but I reviewed the suggestions, implemented them manually, tested them locally, and changed them where necessary.

For Parts A and D, the technical arguments, decisions, conclusions, and reflections were my own. I wrote the content myself and used ChatGPT afterwards to improve wording and grammar for the final report.

Representative prompts included requests such as: “help me understand how Neo4j works”, “explain this Cassandra data model”, “help me understand this error” and “improve the wording of this paragraph without changing its meaning”.

References

1. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
2. Gilbert, S. & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51–59.
3. Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. IEEE Computer, 45(2), 37–42.
4. Chang, F. et al. (2006). Bigtable: A distributed storage system for structured data. ACM Transactions on Computer Systems, 26(2), 1–26.
5. Robinson, I., Webber, J. & Eifrem, E. (2015). Graph Databases (2nd ed.). O'Reilly Media.
6. Vogels, W. (2009). Eventually consistent. Communications of the ACM, 52(1), 40–44.
7. Week 1–7 lecture slides, Advanced Data Management, ECE — University of Thessaly, Spring 2026.